

# NautiKube - Full Report

Generated: 2026-04-15 20:36:34 | Version: dev  
Context: kind-kind-2 | Namespace: all namespaces | Issues: 5

| SEVERITY | RESOURCE       | NAME                       | SCORE | ISSUE                                             |
|----------|----------------|----------------------------|-------|---------------------------------------------------|
| HIGH     | ClusterRole    | cluster-admin              | 70    | ClusterRole has wildcard permissions              |
| LOW      | ServiceAccount | local-path-provisioner-... | 30    | ServiceAccount has no secrets or imagePullSecrets |
| INFO     | Cluster        | CoreDNS                    | 20    | CoreDNS is healthy (2 pods ready)                 |
| INFO     | Cluster        | API Server                 | 10    | API Server is healthy and responding              |
| INFO     | Cluster        | Nodes                      | 10    | All nodes are running consistent version: v1.34.0 |

## Issue Details

#1 -- ClusterRole -- cluster-admin

**Severity:** HIGH  
**Namespace:**  
**Score:** 70  
**Issue:** ClusterRole has wildcard permissions  
**Explanation:**

O ClusterRole concede permissões em todo o cluster usando o wildcard '\*' para verbos, recursos ou apiGroups. Qualquer sujeito (usuário, grupo, ServiceAccount) vinculado a esta role obtém acesso irrestrito a esse tipo de recurso em TODOS os namespaces, violando o princípio do menor privilégio.

Nota: ClusterRoles de sistema integrados (cluster-admin, admin, edit, view) são intencionalmente amplos. Se esta é uma role de sistema, verifique se ela está vinculada APENAS a sujeitos confiáveis e necessários.

- Passos:
- 1. Inspeção o YAML do ClusterRole para identificar quais regras contêm wildcards.
  - 2. Encontre todos os ClusterRoleBindings que referenciam esta role -- esses são todos os sujeitos afetados.
  - 3. Para cada regra com wildcard, substitua '\*' apenas pelos verbos e recursos específicos necessários.
  - 4. Use kubectl auth can-i para verificar se a role restrita tem as permissões corretas.

**Remediation:**

```
[1] kubectl get clusterrole cluster-admin -o yaml
[2] kubectl get clusterrolebindings -o wide | Select-String 'cluster-admin'
```

#2 -- ServiceAccount -- local-path-provisioner-service-account

**Severity:** LOW  
**Namespace:** local-path-storage  
**Score:** 30  
**Issue:** ServiceAccount has no secrets or imagePullSecrets  
**Explanation:**

O ServiceAccount não tem Secrets ou ImagePullSecrets associados.

CONTEXTO IMPORTANTE: Desde o Kubernetes v1.24, ServiceAccounts não criam mais automaticamente Secrets de token de longa duração -- isso é intencional e uma melhoria de segurança. Para ServiceAccounts de sistema no kube-system (controllers, scheduler, etc.), esta descoberta é ESPERADA e benigna; nenhuma ação é necessária.

Só aja se TODAS estas condições forem verdadeiras:

- O ServiceAccount é usado por pods em um namespace gerenciado pelo usuário.
- Esses pods precisam fazer pull de imagens de um registry de contêiner PRIVADO.
- Os pods estão realmente falhando com erros de ImagePullBackOff.

Correção (apenas se necessário):

1. Crie um imagePullSecret com suas credenciais de registry.
2. Aplique patch no ServiceAccount para referenciar esse secret.
3. Verifique se os pods agora conseguem fazer pull das imagens com sucesso.

#### Remediation:

```
[1] kubectl get sa local-path-provisioner-service-account -n local-path-storage -o yaml
[2] kubectl get pods -n local-path-storage -o wide | Select-String
'local-path-provisioner-service-account'
```

### #3 -- Cluster -- CoreDNS

**Severity:** INFO  
**Namespace:** kube-system  
**Score:** 20  
**Issue:** CoreDNS is healthy (2 pods ready)

### #4 -- Cluster -- API Server

**Severity:** INFO  
**Namespace:**  
**Score:** 10  
**Issue:** API Server is healthy and responding

### #5 -- Cluster -- Nodes

**Severity:** INFO  
**Namespace:**  
**Score:** 10  
**Issue:** All nodes are running consistent version: v1.34.0